Foundations of Data Structures

Arrays, Lists, Trees & Algorithmic Efficiency

Sample coursework material — for product demonstration purposes.

Chapter 1 — Why Data Structures Matter

A data structure is simply a way of organizing data in memory so that it can be accessed and modified efficiently. The same information can often be stored in several different ways, and the choice of structure has a direct impact on how fast a program runs and how much memory it consumes.

There is no single 'best' data structure — each one makes trade-offs between the speed of different operations: reading an element, inserting a new one, deleting one, and searching for a value. Learning these trade-offs is the foundation of writing efficient software.

Chapter 2 — Arrays: Fixed, Fast, Contiguous

An array stores elements in a single, contiguous block of memory, each accessible by a numeric index. Because the computer can calculate the exact memory address of any element from its index, reading or writing any element takes constant time, written as O(1), regardless of how large the array is.

This speed comes at a cost: inserting or removing an element in the middle of an array requires shifting every subsequent element to keep the memory contiguous, which takes time proportional to the array's size, or O(n). Arrays also have a fixed size in many languages, meaning that growing beyond the original capacity requires allocating a new, larger block and copying every element over.

Chapter 3 — Linked Lists: Flexible but Sequential

A linked list stores each element in its own node, along with a pointer to the next node in the sequence. Unlike an array, these nodes do not need to sit next to each other in memory — the pointers do the work of keeping the sequence connected.

This makes insertion and deletion very cheap once you already have a reference to the right spot: rewiring a couple of pointers is an O(1) operation, with no need to shift other elements. The trade-off is that linked lists have no direct indexing — finding the 500th element means walking through all 499 nodes before it, an O(n) operation, and each node needs extra memory to store its pointer.

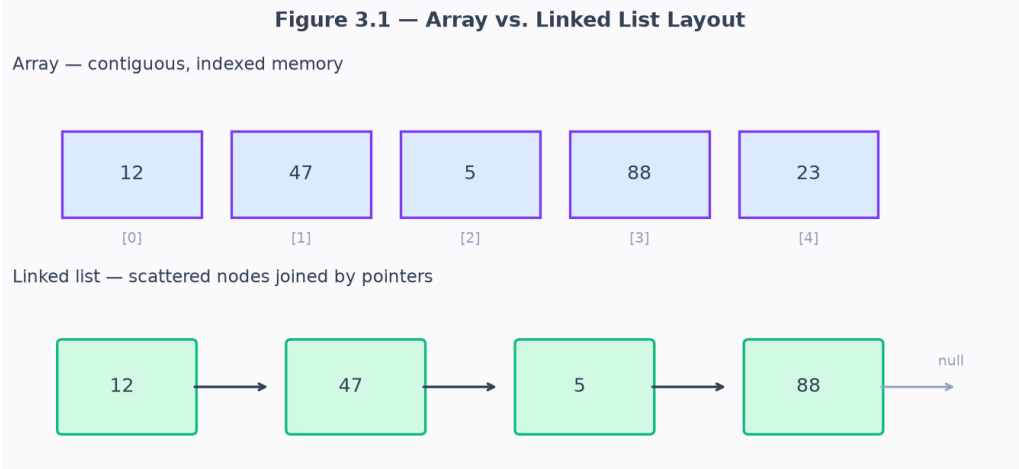

Figure 3.1 — An array stores values contiguously and indexed; a linked list scatters nodes connected by pointers.

Chapter 4 — Stacks and Queues

Stacks and queues are restricted forms of a list that control the order in which elements can be added and removed. A stack follows Last-In-First-Out (LIFO) order: the most recently added element is the first one removed, like a stack of plates. Stacks are the mechanism behind function call tracking, undo history, and expression evaluation.

A queue follows First-In-First-Out (FIFO) order: the first element added is the first one removed, like a line at a checkout counter. Queues are used to manage tasks processed in the order they arrive, such as print jobs or requests waiting to be handled by a server.

- Stack operations: push (add), pop (remove most recent) — both O(1).
- Queue operations: enqueue (add), dequeue (remove oldest) — both O(1).

Chapter 5 — Trees and Binary Search Trees

A tree organizes data hierarchically: each node has a single parent (except the root) and any number of children. Trees naturally represent hierarchical relationships, such as a file system's folders or an HTML document's structure.

A binary search tree (BST) is a tree in which every node has at most two children, and is arranged so that every value in a node's left subtree is smaller than the node, and every value in its right subtree is larger. This ordering means that searching, inserting, or deleting a value takes roughly O(log n) time in a balanced tree — dramatically faster than scanning a list — because each comparison eliminates half of the remaining nodes from consideration.

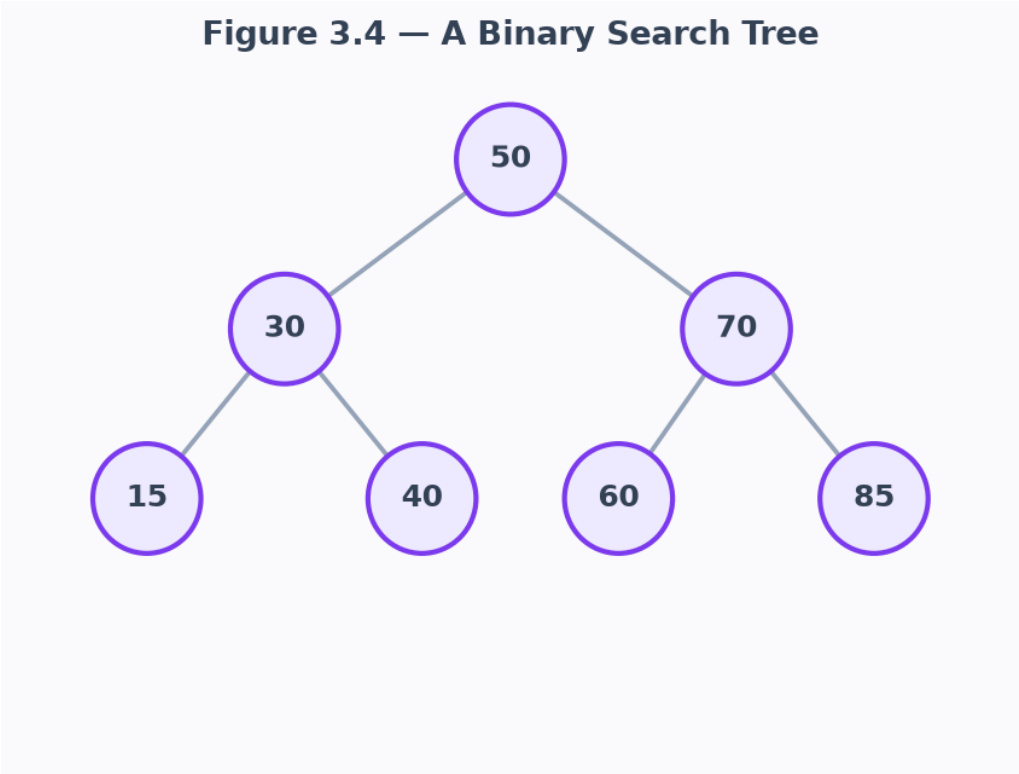

Figure 3.4 — A binary search tree. Every left child is smaller than its parent; every right child is larger.

Chapter 6 — Big-O: Talking About Efficiency

Big-O notation describes how the running time (or memory use) of an operation grows as the size of the input, n, grows — it is a way of comparing algorithms independent of any particular computer's speed.

O(1) means an operation takes the same amount of time no matter how large the input is, like array indexing. O(log n) means the time grows very slowly as input grows, like a balanced binary search tree lookup. O(n) means the time grows in direct proportion to the input, like scanning a linked list. Understanding which category an operation falls into is what lets engineers predict whether a piece of code will still run quickly when the input grows from a thousand items to a billion.